(**a preprint version to be submitted to preprint servers**) by
Shymaa M. Arafat
Associate Professor in Computer Science & Engineering (Independent)
Shar.academic@gmail.com, Shymaa.Arafat@gmail.com

Now We Have Enough Evidence to Go to Court- A (2 Devices) and (3+ Receipts) Poll Station E-Voting system

Abstract. The constitutional German court finds cryptographic proofs "*somewhat untenable as a verification*" since they need an expert testimony; hence Germany, and many other countries, do not allow e-voting in their political elections. In this paper, we propose an in-booth e-voting solution that allows voters to challenge their individual votes in court and allows judges to verify by themselves without calling cryptographic experts. The proposed system[1] uses 2 devices; the first device authenticates the voter and prints a receipt with a unique cryptographic ID code (*HBID*) for each voter. The second is a voting device that permits successive multiple voting and prints a receipt for each voting attempt; the device then prints a sealing receipt for poll workers with enough information to identify the voter's final receipt. We combine the Benaloh challenge idea and the multiple voting option to deceive vote coercers/buyers and detect malicious voting devices at the same time, without losing our court verifiability target. In addition, we suggest to hide the printed vote inside a simple function of the vote & *HBID*, as an optional extra safe guard to protect from coercion by poll workers (who will take the distinguishing receipt); although it is supposed to be fast & easy for the average person to get the vote from the simple function in the receipt, it would be time consuming and noticeable if poll workers did it inside the poll station for all voters. The proposed system also publishes per booth summary reports every fixed interval to serve as check points and to be used by Risk Limiting Audits (RLAs) and Zero Knowledge (ZK) queries as well. Finally, election result is published in a Public Bulletin Board as records (*booth, time, vote, no of votes*) to anonymize it from adversaries.
Keywords: E-Voting, tracking codes, vote coercion, vote buying, RLAs.

1 Introduction

E-voting techniques provide many helpful tools for verifiability and fraud/manipulation detection; however, as [1] put it "*for voters that do not verify or whose verification failed, we make no guarantees*"; something that is strikingly important when most e-voting statistics [2,3] show that verification ratios hardly reach 10%. If non verifying voters have only themselves to blame, e-voting systems should study the impact of those possibly wrong votes on universal verifiability; what if those votes were all manipulated, especially in systems that do not check a random sample of votes through *Risk Limiting Audits* (*RLA*).

[1] The idea of this paper was presented as a poster in a conference that reviewed posters based on abstracts only; https://anonymous.4open.science/r/anonymous_citation-E37B/CourtVerify_anonymized.pdf. The conference did not publish a book of abstracts for the posters; hence, the author finds it ethically appropriate to publish a full paper extension of the poster

Another problematic issue is what the constitutional German court stated about cryptographic proofs being "*somewhat untenable as a verification*" , [4,5], since they require expert testimony; an argument that would be further emphasized in developing countries, [6], which scores less in both the availability of highly qualified cryptographic experts and public trust in voting systems.

In this paper, we target an e-voting system that, while still being able to detect manipulations and avoid vote buying/coercion, ***gives the opportunity for any voter, or a group of voters, to challenge the voting system in court***; a losing candidate can gather a sample of his/her supporters and go to court to check their votes. Our proposed design aims to achieve what we call complete and sound ***court verifiability***; defined as: *the court ability to decide the integrity of election, without expert knowledge, by using receipts with tracking codes and public bulletin boards*.

We consider the main contribution of this paper is deploying Benaloh challenge to achieve multiple purposes; by printing a receipt for every trial, it provides evidence receipts for courts and extra receipts for deceiving vote coercers/buyers in addition to its original purpose of testing the voting device. The following section, section 2, explains the philosophy behind the most important design decisions; how we have chosen the suitable ingredients to our targeted goals from the available literature. Then, rest of the paper is organized as follows; section 3 describes the system architecture and the voting steps, while section 4 presents exhaustive system analysis. Then section 5 shows the latest progress in a WIP (Work In Progress) automated formal verification of the proposed system using ProVerif2.05; the attempt was started with the help of AI. Finally, section 6 summarizes the work with concluding remarks.

2 Motivation Behind Main Design Decisions

-We preferred in-booth e-voting to avoid infrastructure deployment problems and/or cyber security risks.

-We also used two devices to separate the voter authentication step from the voting step, where the two machines should be isolated (air gapped) with a removal memory card; *distributing trust* is a main design strategy that can be inferred in most system choices.

-We use the first device to authenticate voters without repetition; i.e., checking this authenticated ID belongs here (in this booth) and has not been here before[2] to avoid repeated or absent voting. The first device is also responsible for generating and printing a per voter tracking code, we call ***HBID*** *(Hiding & Binding ID)*, that will appear in all receipts.

[2] This could be done in different possible ways; for example, scanning IDs/electronic IDs, biometric checks or both. There are problems associated with biometric devices, like those happened in Afghanistan elections [7], but they are the most used to avoid absent voting from Africa [6] to India [8] and even in less crowded polls like UAE [9]. Hence, we assume proper functionality and suffice with consistency checks for now.

-We do not let the voting machine use the original vote ID, or generate the HBID itself, to distribute trust between 2 isolated machines. Could have been an alternative design option to follow the Swiss Post, [10], methodology of sending HBIDs using regular mail; however, this would increase the error probability of possible multiple voting in different booths and will also remove away the benefits of time difference checks between the two machines.
-The second voting machine deals only with *HBID* as the new identity after reading a printed passing approval generated by the first machine. We introduce *a modified Benaloh challenge* [11,12] approach that allows multiple intermediate voting with receipts, to both detect a malicious voting device (the device has no way of telling whether this receipt is final or not) and at the same time have an expired receipt to show any vote coercer/buyer.
-Valid voting options should *include "**boycott**" and "**reject all**"*[3] for complete protection from coercers.
-To allow deceiving adversaries and prevent deceiving judges at the same time, we print (in different color to avoid mistakes) *an extra sealing receipt for poll workers* to keep (includes final time and number of multiple votes); the sealing receipt should be submitted in court to either assert or falsify a voter's claim (could also be used for Risk Limiting Audits RLAs).
-We also offer an extra optional step to waive the assumption of "trusted poll officers"; in *the final poll workers receipt*, the second device *does not print the clear vote, but a simple function of the vote and the tracking code*. The function should be simple enough to be reversible in 1-2 mins by voters and judges, yet it would be time consuming and publicly noticeable if a poll worker dedicated 1-2 mins/voter to calculate the vote from the function before allowing the voter to sign and leave[4].
- Letting voters throw the sealing receipt (with a clear vote then) in a ballot box, without poll workers interference, might jeopardize both system usability and integrity; voters may throw a wrong receipt, or even not throw it at all, whether mistakenly or deliberately. In our proposed solution, ***the sealing receipt is the election authority's responsibility***. If they claimed in court that it is missing, then they are the ones to be blamed/doubted; while if voters were to throw it in the ballot box, we have no way of telling who is malicious or responsible.
-Finally, proving non-voting starts with a *SNARK* (*Succinct Non-Interactive Argument of Knowledge*) supporting non-inclusion proofs, then follows a court convincing methodology as detailed in section 4.4.1 (absent voting).

[3] Sometimes coercers want voters to boycott elections or invalidate their votes, [7,13]; this way votes can still deceive them and vote.

[4] In most real-world cases, poll officers are assumed trusted; only audited by independent observers and/or candidates' representatives. The linear function is just an extra barrier from coercion, we believe complete protection is not possible here; they can force you to show any receipts/QR-codes, they can stand behind you while voting, ...etc. *If you are counting on auditors' presence, then why not count on them regarding the linear function in the receipt?*

3 System Architecture & Voting Steps

The architecture of the system is described in Fig.1, follows is a further elaboration.

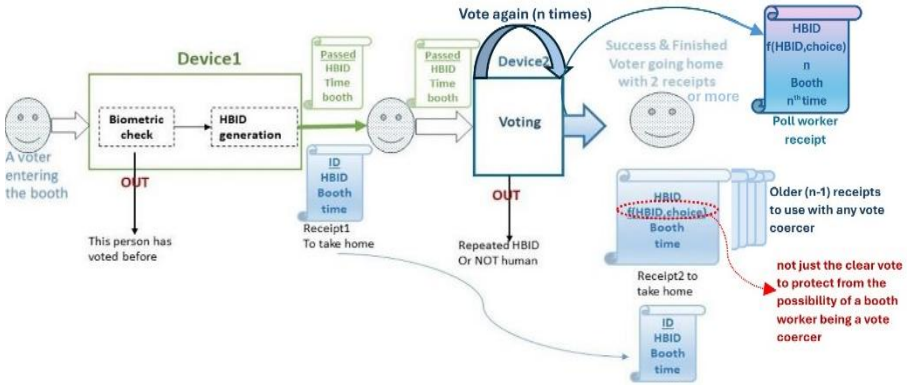

Fig.1. The voting process in the proposed system

-**The generated *HBID*** must be 1)hiding, 2)binding, and 3)unpredictable from the voter ID with overwhelming probability. If *HBID*s were to have collisions the system would collapse; if voter ID could be revealed or predicted from it, coercers can code their calculations prior to election. Hence, it is a strong cryptographic hash function (possibly post-quantum) that includes randomization; same analogy of *randomized leader election* [14].

-**The linear function printed in receipt2** holds a trade-off between usability and Poll officers' trust. It could be as simple as flipping a bit position in HBID that is function of the voter's choice (not the exact choice index to avoid border bits, say middle bit of choice index *(ch_indx*); i.e., if HBID is *n* bits and there are $m<n$ choices in this election, then an example function would be:
Code[i]=HBID[i];

F(ch_indx, HBID) = Complement (Code [(*n-m)*/2 +*ch_indx*]);

3.1 Voting Steps

1. When entering booth, the voter submits ID to device1 and authenticates he/she is a registered voter in this booth who hasn't voted yet.[5]
2. Device1 generates a new *HBID*, appends an encrypted tuple *(ID, HBID, bio, time, booth)* to its memory card and aggregates data (before encryption) in a Merkle/Verkle Tree.

[5] Example procedure could be to perform a biometric check, then starts some dialogue with the machine to prove humanity (to detect previously collected photos or any offline material inserted in the machine).

3. Device1 prints *a timed passing ticket* to be scanned by device2, and a ***receipt1*** for the voter to keep containing *ID, HBID, Time, Booth*. The passing ticket contains the HBID to be scanned by device2 to avoid human entering errors.
4. The voter goes to device2; submits his/her passing ticket; device2 checks that this HBID is not already in its memory.[6]
5. Device2 shows the choices for the voter; the voter clicks on the desired choice, then device2 shows on screen a preview of a *timed receipt with HBID and* the simply coded choice *f(choice, HBID)*; the print preview screen should show how f(choice, *HBID*) is calculated for the voter to understand the receipt meaning. If vote is confirmed as correct, device2 prints it and appends its info to the memory card inside it.
6. The voter may repeat step 5 several times, and *every receipt2 will contain a vote counter*. After confirming this is the final vote, device2 flags its record in memory and prints timed ***receipt3*** to be handed to poll workers and kept in a box.

3.2 At Checkpoints and After Voting Closes

The following steps, as depicted in Fig.2, should take place after the voting closes; also, every fixed interval to detect and handle errors earlier and limit the impact of an error to only one interval in one booth.

7. Each device prints cryptographic checksums of its data for auditors and poll officers to *check the consistency of data from both devices and consolidate them in **a summary report***. The report may contain the number of voters, confirming no repeated *HBID*s, cryptographic commitment of the list of *HBID*s (ex.: Merkle/Verkle root); could also contain some more checks from device1 like biometric features or Male/Female ratio. Then all auditors and poll officers *sign the summary report either manually in a public document or by their public keys to a public Blockchain* containing **(booth ID, date & time of session, no of voters, no of votes,...)**; as a more distribution of trust, the number of voters should be also checked against handwritten signatures[7] totals. Risk Limiting Audits (*RLA*s) can be performed by cryptographically verifying the correct corresponding data of sampled poll workers receipts in both devices[8]. If errors were found, they should be listed too.
8. Poll workers remove memory cards from both devices and deliver them along with the sealing receipts box to the election authority.
9. The election authority aggregates all memory cards in a voting server which will check IDs are not repeated in any other booth, match the 2 cards record by

[6] It is possible to conduct a second humanity proof dialogue here to prevent automated voting.

[7] So that, if the two machines colluded to have consistent fake summary reports, they will differ from the handwritten registry.

[8] The cryptographic ZKP will prove the HBID existence in device1 and the number of votes for that voter in device2; another example is checking that (time on device1 is < time on device2) with reasonable difference.

record[9], check times are consistent, check consistency with subtotals in summary reports,...etc. We assume the remaining steps will be performed by the election authority as in most voting systems and not the concern of this paper.

10. When the voting server completes the counts, the election result is uploaded to a public bulletin board in the form *(booth, time, choice, no_of_votes)*; we escape from vote buying/coercion depending on the fact that it is impossible for 2 votes to have the same exact time and booth. Hence, any voter can identify his/her vote in the Public Bulletin Board, while adversaries can at maximum calculate the ratio of those who deceived them (when they cannot find recorded time and booth pairs in the receipts they have). It is true that coercers and/or vote buyers can use more sophisticated ways to identify those who deceived them, however, this can happen only after the election ends; see section 4.5.6 for more details

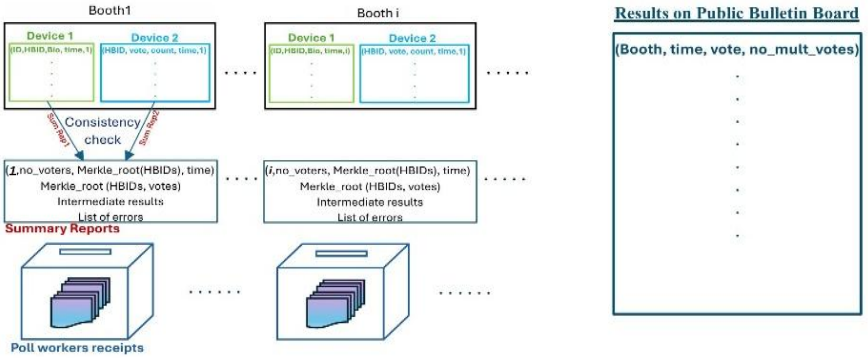

Fig.2. Data and Summary reports from booths periodically and after the election.

4 Protocol Analysis

4.1 Cost of Devices & System Assumptions

• The first device is assumed to function properly with a high-resolution camera, scanner or both; the second device needs a scanner to read the HBID.
• Both devices must be completely isolated from any network and contain a removable memory card.
• ***We assume that the printed receipts can never be forged***, through a special paper for example or maybe some kind of automatically printed seals and/or watermarks.
• ***We assume the voting machine can never print two sealing receipts for the same HBID***.
• The properties of HBID are a primary assumption.

[9] To avoid the possibility of an adversary getting access to the 2 cards (or their records) somehow and performs the matching as well to know the corresponding identity of each vote, each device maybe assigned an encryption key (by the election authority) to encrypt its record.

• Finally, the *Election Authority* (*EA*) trust is controlled by the published summary reports; subtotals must be consistent with totals, and fault intervals can be identified.

4.2 Trust Assumptions & Malicious Devices

Table 1 summarizes how the proposed e-voting system handles the possibility of each of its components being malicious, then follows the details.

Malicious Entity	Defense
Device 2	Benaloh Challenge
Device 1 & Device 2	Summary reports, RLAs, consistency checks
Public Bulletin Board (PBB)	Clash attacks and alike not applicable; PBB doesn't know the checking voter (identify with time & booth)
Election Authority (EA)	Trusted in privacy, voters' receipts+summary reports protect integrity
Poll Officers	The linear function + auditors + RLAs

Table 1: System defense against malicious entities

4.2.1 Malicious Device2

Benaloh challenge and the public bulletin board are first line defenses to detect malicious voting devices, and the proposed system applies both; many other attack scenarios, like the two machines colluding together, can be detected by RLAs and summary reports consistency checks as in Figure 2.

4.2.2 Public Bulletin Board Attacks

In our proposed system the PBB has no need to know the voter identity before showing the results, which excludes the possibility of showing different views to different voters; i.e., the attacks in [15,16] are not applicable here, since voters cannot be deceived to check the same entry; they can't possibly print from the same device in the same booth at the same time. Finally, manipulations like adding/deleting/altering votes will contradict with subtotals in the summary reports and sealing receipts count.

4.2.3 Voting Servers & Election Authority

Although *EA* is the only one trusted to know the vote and the clear ID, *EA* servers are not trusted alone to alter the results; each voter has receipts, and also each (voting location, booth,…) is embedded in a published signed summary report. In addition to cryptographic functions, the summary reports contain the number of voters and the subtotals for each candidate in the clear to be readable by judges.

4.2.4 Poll Officers

In most elections, poll workers trust is controlled by the presence of auditors from election observation organizations and candidates presentative. Here, in the proposed system, the linear function that hides the vote makes large scale vote coercion and/or privacy violation more noticeable by election observers; in addition, the check points

summary reports (whether manual or digital signatures were used) distribute trust among all signers.

4.3 Dispute Resolution

The researchers in [17] tried to abstract a model for dispute resolution in any e-voting system, [16/sec. IV]; it all comes down to the complaining voter claim versus the Election authority response. Table 2 summarizes possible scenarios in the proposed e-voting system and how the court should act in each case; details are provided in the following subsections.

Voter's Claim / **EA Response**	"***I can' find my vote in PBB***" Or "***I didn't vote, but Non-inclusion Proofs says I did***"
EA brings a sealing receipt and voter's whole record with signature/authentication supporting their claim	Malicious Voter
No sealing receipt supports EA claim (contradicts or does not exist)	Falsify election in this interval in this booth
EA claims lost receipt	Their responsibly, rule for voter
EA doesn't show up in court	Assumed guilty, rule for voter
EA claims "an error in non-inclusion proofs, we agree this voter did not vote"	Perform a manual recount in the specified booth & interval

Table2: Court Judgment according to different Election Authority (EA) Reactions

4.4 Protocol Robustness Against Known Attacks

4.4.1 Absent Voting

Individual voters who did not vote, or doubting parties who suspect there has been absent voting, can start by first checking non-inclusion proofs from the cryptographic commitment in the summary report (Verkle or Merkle value). If they find a contradiction, they can go to court; i.e., the ZKP is used only as counter example evidence to start the investigation.

The court then asks the authorities to bring the final receipt and the whole voter's record; there is more than one possibility:

1- The authorities bring the required data accusing the voter of false claims. Then both the vote timing and the handwritten signature should be checked; for example, if the voter can prove he/she was elsewhere during this time then the vote is forged.

2- The authorities claim a wrong ZKP implementation saying they agree the voter did not vote, but this doesn't reflect a problem in the election results.
   - Since the summary reports can limit the doubt only to the interval between 2 checkpoints, the court may ask for their full records to perform a limited manual check and count.
   - Also, the court may call a random sample of this interval's voters to come with their receipts for a manual consistency check, where RLAs rules may be applied.

Here, the worst-case court decision is to falsify the doubted intervals (not a complete election) and could vary on a case-by-case basis. The decision could depend on the number of complaining voters, the error ratio, the uncertainty of answers, …; the court may also ask a cryptographic expert on this one (why the ZKPs gave wrong results, what are the possible reasons given the code in hand, ...) but only as an auxiliary help.

4.4.2 Double Voting

A deviation from the summary reports may happen due to detecting a repetition of the same HBID in different booths. The voting servers, by holding all the data, can locate the complete records with the time and biometric checks to trace the error and discard those votes. However, this situation is nearly impossible to happen; it could result only from a collision in the generated HBID which violates an original system assumption. If an adversary managed to get access to device2 with an old passing ticket after the voter leaves, this should be rejected by device2; it is after printing a previous sealing receipt for this HBID, and for passing the time limit between the two devices. If device2 was malicious and allowed it, this will be discovered through the summary reports. A voter who managed to vote in two different booths, although should be rejected by device1, will get two different HBIDs.

4.5 Protocol Robustness According to Performance Metrics

4.5.1 Individual Verifiability

Individual verifiability is achieved through voters' receipts and the public bulletin board; every voter can search for the time recorded in his/her final receipt.

4.5.2 Individual Court Verifiability

Any voter, or a group of voters can be gathered by a losing candidate (or a winning candidate defending his/her position), to verify or falsify their true votes in front of court. The court has 3 main sources for the truth: *voter's receipt*, *Election Authority (EA) receipt*, and the *Public Bulletin Board (PBB)*; cryptographic SNARKs from the summary reports (Merkle/Verkle of final/multiple votes) may be used only as an asserting clue. A nearly similar argument to that provided in absent voting, Table 2, applies here as well

Soundness of Court Verifiability

•A malicious voter falsely claiming he/she did not vote, will not succeed in deceiving the judges as explained in absent voting.
•Similarly, a malicious voter cannot falsely claim a fraud vote to disrupt the election if judges can perform the simple function calculation by themselves then compare it with the EA receipt and PBB records. Even if EA refused to respond, judges should search the PBB for the (*time, booth, vote count*) in the voter's receipt and will not find it but will find a slightly higher time and count; the time difference would be less than the average difference between 2 voters and proportional to the difference in vote count. Hence, the judge will reject the voter's claim that this is the final receipt; a non-inclusion proof from the corresponding summary report could be used as auxiliary assuring evidence.

Completeness of Court Verifiability
If *EA* colluded to properly alter the *PBB* record of an honest voter, then judges must call *EA* to submit the corresponding sealing receipt; since we assume sealing receipts cannot be printed more than once for the same *HBID*, the truth will be revealed. If *EA* claimed losing their receipt, then it is their responsibility; they are to be assumed malicious. The court may rule at once or use a cryptographic *SNARK* from the corresponding summary report for assurance only.

4.5.3 Universal Verifiability
Achieved through summary reports, biometric checks, sealing boxes count, RLAs, and zero knowledge proof queries to the server. We believe those results will be sound in court too, since the subtotals were published periodically in an immutable blockchain and handwritten signed records. Extra queries are provided by the election authority and cannot be forged for a certain booth without the collusion of both auditors and polling officers. In the worst-case scenario, poll workers receipts could be manually checked one by one against the public bulletin board data, and errors traced back with booth number and voting time.

4.5.4 Universal Court Verifiability
If a losing candidate gathered a group of verifying voters that equals or exceeds the RLA sample size, the court can call for a manual recount if votes were proved to be wrong.

4.5.5 End-to-End Verifiability
The receipts prove votes are "*casted as intended*", while the public Bulletin Board assures that votes are "*recorded as casted*" and consequently "*counted as recorded*".

4.5.6 Coercion Resistance
The proposed system is completely robust against vote buying/coercion from external players by handing them an old receipt; even if their purpose was to boycott the election or choose "reject all", our system provides these options as valid choices. The public Bulletin Board is not a risk here, since it does not include the ID or the HBID values. If the threat came from the poll workers, the hiding function provides a

reasonable protection at election time and HBID is assumed to be irreversible (hiding); i.e., they cannot retrieve the IDs of undesired votes to trace them afterwards.

We admit that there're ways the coercer can identify non-complaint voters (but not their true vote); if kept the receipts or their photos and wrote the voter ID/name on each receipt, or if simply if targeting a certain specific voter. For the same argument regarding poll-workers and receipt3, this cannot be realistic when vote buyers/coercers hang out near poll stations to pay voters (a typical seen in developing countries elections).

However, this can happen in a more technically sophisticated methodology, where voters are asked to register their IDs on a certain site and upload a copy of the receipt to get paid; the PBB search could be automated then to produce a list of disobeying voters.

Still, in both cases, the coercer will not be able to find out before the election results are announced; i.e. if people are preparing for some kind of election-style revolution, they can deceive coercers till after the fact.

5 WIP: Testing the Proposed System Using ProVerif

Automated verification tools, like ProVerif [18] and Tamarin [19], have become a significant research tool for testing e-voting systems in the last few years; [20] is a PhD from Luxembourg University, and ProVerif tool is rooted in Inria where one can trace more research on its improvements [21,22]. In this section, we just demonstrate the applicability of our proposed e-voting system to being tested by automated formal verification tools using ProVerif2.05 as the most common tool; an attempt that we started with the help of AI [23] and plan to complete in our future work.

-We first wrote the system main skeleton as[10]:

[10] we called the types "time1" and "choice1" because the words "*time*" and "*choice*" are keywords used by ProVerif2.05

```
(* Types *)
type id.
type hbid.
type choice1.
type time1.
type booth.
type fval.
type count.

(* Functions *)
fun H(id, bitstring): hbid.
fun F(hbid, choice1): fval.

(* Events *)
event cast_final(hbid, choice1, time1, count, booth).
event voter_receipt(hbid, choice1, time1, count, booth).
event sealing_receipt(hbid, fval, time1, count, booth).
event pbb_entry(booth, time1, choice1, count).
event ea_receipt(hbid, fval, time1, count, booth).
event judge accepts(hbid, choice1, time1, count, booth).
```

-For testing, as a start, the most important feature in our system, ***court verifiability***, we used two queries

```
(* Queries *)
(* Soundness: judge_accepts implies a unique cast_final *)
query h: hbid, v: choice1, t: time1, b: booth, n: count;
      inj-event(judge_accepts(h, v, t, n, b)) ==> inj-
event(cast_final(h, v, t, n, b)).

(* EA cannot fabricate a receipt without a cast_final *)
query h: hbid, fv:fval, v: choice1, t: time1, b: booth;
      inj-event(ea_receipt(h, fv, t, n, b)) ==> inj-
event(cast_final(h, v, t, n, b)).
```

-We first tried a completely honest process, and the complete code as in [24] was run successfully.

```
(* process: one honest run *)
process
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
  new T  : time1;
  new B  : booth;
  new C  : choice1;
  new N : count;
  let FV = F(HB, C) in

  (* Ground truth and evidence events *)
    event cast_final(HB, C, T, N, B);
  event voter_receipt(HB, C, T, N, B);
  event sealing_receipt(HB, FV, T, N, B);
  event ea_receipt(HB, FV, T, N, B);
  (* Judge/EA decide to accept this triple *)
  event judge_accepts(HB, C, T, N, B);
  event pbb_entry(B, T, C, N);
  0
```

-Then, we tried to expose receipts to attackers

```
(*exposing receipts to attackers*)
free net : channel.

process
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
  new T  : time1;
  new B  : booth;
  new C  : choice1;
  new N : count;
  let FV = F(HB, C) in

  event cast_final(HB, C, T, N, B);
  event voter_receipt(HB, C, T, N, B);
  out(net, (HB, C, T, N, B));  (*voter shows receipt on a public channel*)

  event sealing_receipt(HB, FV, T, N, B);
  event ea_receipt(HB, FV, T, N, B);
  out(net, (HB, C, T, B));           (*EA's sealing receipt visible too*)

  event pbb_entry(B, T, C, N);
  out(net, (B, T, C, N));            (* PBB is public by design *)

  event judge_accepts(HB, C, T, N, B);
  0
```

The final code as in [25][11] also ran successfully; the last screen is shown in Fig.3, while the whole output in [26].

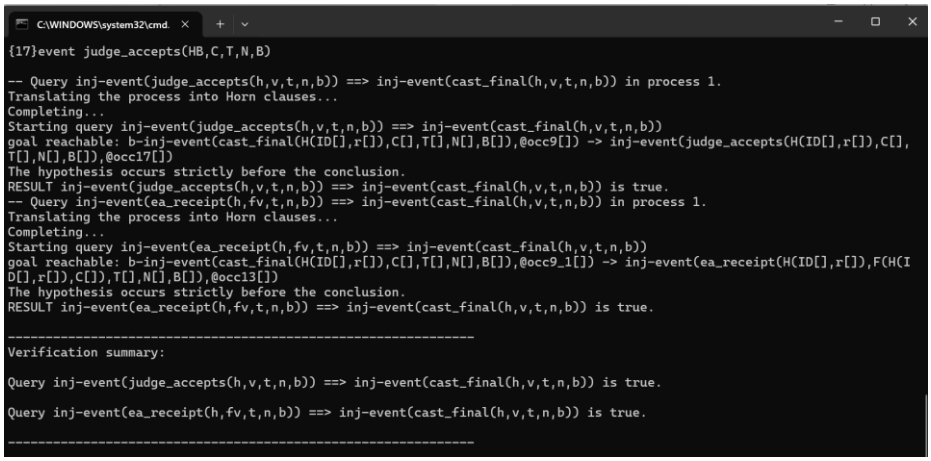

Fig.3 Output of Proverif2.05 code run

Finally, we leave the rest of automated formal verification steps as future work.

6 Summary & Future Work

All reported statistics show that election QR codes are rarely verified (≤10%), [2,3], yet there is always a persistent minority of losing candidates who doubt the results and may challenge the election authority in court. An honest judge is trapped between two fears; to be deceived by complicated terms out of his/her area of expertise, or to fall for hypothetical arguments and falsify the election (wasting money and effort) without solid proof.

In this paper, we proposed a new feature that even paper ballot voting systems lack; ***court verifiability*** is the ability of verifying votes in court without expert knowledge. Then, we presented an E2E verifiable voting system that guarantees court verifiability through two final receipts which could be submitted in court and accepted as evidence. The system also prints expired voting receipts that could be used to deceive vote buyers/coercers; however, if a voter or a losing candidate tried to deceive the court with the same trick, the system provides an extra distinguishing receipt that is kept by the Election Authority and can be brought to court in dispute.

We simply proposed a new middle spot between traditional tradeoffs in e-voting systems that we believe is satisfying, then we supported our claim with exhaustive analysis. We also introduced an interesting extra beneficial use of the classical Benaloh challenge; to deceive vote coercers with expired voting receipts.

[11] Since this is a WIP, another file "*court_final.pv*" will also be kept in the same folder to hold the newest version; where at the time of writing "*court_final.pv*" is an exact copy of "*court3.pv*" in [25] that contains the displayed code (the current running version).

To clarify the main idea, some details may have been abstracted like the hash function, the RLA technique used, and the exact consistency checks; also, some details could be altered according to system needs like the exact fields in the summary report which makes the protocol a perfect candidate for testing using automated formal verification tools.

Learning & practicing automated formal verification tools in general and using them to test the proposed system is a WIP we started its first steps for testing court verifiability using ProVerif2.05; to be continued as a future work.

Acknowledgements

All cited references are open source, and most acquired e-voting knowledge came from free material available on the internet. The Automated Formal verification steps in the Appendix were done with the help of AI (Copilot). Finally, gratitude is always there for those who taught us to be persistent on our research goals and always seek the optimal; graduation faculty professors and postgraduate supervisors.

References

1. Kristian Gjosteen et al., "*Coercion Mitigation for Voting Systems with Trackers: A Selene Case Study*", E-Vote-ID 2023, LNCS14230, pp.69–86; https://doi.org/10.1007/978-3-031-43756-4_5
2. Nicole Goodman et al.," *Verifiability Experiences in Ontario's 2022 Online Elections*", E-Vote-ID 2023, LNCS14230, pp.87–105,2023; https://doi.org/10.1007/978-3-031-43756-4_6.
3. https://www.valimised.ee/en/archive/statistics-about-internet-voting-estonia; last accessed 28/11/2025.
4. NDI: *The constitutionality of electronic voting in Germany*(2019), downloaded 1/12/2023; https://www.osetinstitute.org/blog/electronic-voting-banned-in-germany
5. Prashant Agrawal et al., "*OpenVoting: Recoverability from Failures in Dual Voting*", E-Vote-ID 2023, LNCS14230, pp.18–34; https://doi.org/10.1007/978-3-031-43756-4_2.
6. Hina Bent Haq et al., "*End-to-End Verifiable Voting for Developing Countries- What's Hard in Lausanne is Harder Still in Lahore*", arXiv:2210.04764v1, [cs.CY], Oct 2022.
7. K. Panahi et al, "*But they have overlooked a few things in Afghanistan: An Analysis of the Integration of Biometric Voter Verification in the 2019 Afghan Presidential Elections*", USENIX SS24, https://www.usenix.org/conference/usenixsecurity24/presentaation/panahi
8. https://www.brookings.edu/articles/indias-electoral-democracy-how-evms-curb-electoral-fraud/, last accessed 12/5/2025.
9. https://www.biometricupdate.com/202309/fully-digital-election-with-biometric-remote-voting-planned-for-united-arab-emirates; last accessed 2/2/2026.
10. Switzerland Internet Voting System, Swiss Post, https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master
11. Wojciech Jamroga, "*Pretty Good Strategies for Benaloh Challenge*", E-Vote-ID 2023, LNCS 14230, pp.106–122, 2023; https://doi.org/10.1007/978-3-031-43756-4_7.
12. Josh Benaloh, "*Verifiable Elections Tech: How Voters Can Independently Verify their Votes are Accurately Counted*", UCYB, Nov 2023, https://youtu.be/Xl9lxMETy2M

13. Johannes Müller, Balázs Pejó, and Ivan Pryvalov, "*DeVoS: Deniable Yet Verifiable Voting Updates*", In proceedings of Privacy Enhancing Technologies Symposium 2024, Vol: 2024, Issue:1, pages:357-378, https://doi.org/10.56553/popets-2024-0021, https://www.researchgate.net/publication/377038889_DeVoS_Deniable_Yet_Verifiable_Vote_Updating
14. https://a16zcrypto.com/posts/article/leader-election-from-randomness-beacons-and-other-strategies/; last accessed 2/2/2026.
15. Lucca Hirschi, Lara Schmid, and David Basin, "*Fixing the Achilles Heel of E-Voting: The Bulletin Board*", IEEE CSF, 2021; https://ieeexplore.ieee.org/stamp/stamp.jsp?tp-&arnumber=9505201; https://github.com/LCBH/BulletinBoard-CSF21
16. Olivier Pereira and Dan Wallach, "*Clash Attacks and the STAR-Vote system*", E-Vote-ID 2017, LNCS10615, pp.228–247; https://doi.org/10.1007/978-3-319-68687-5_14.
17. D. Basin, S. Radomirović and L. Schmid, "*Dispute Resolution in Voting*," 2020 IEEE 33rd Computer Security Foundations Symposium (CSF), Boston, MA, USA, 2020, pp. 1-16, doi: 10.1109/CSF49147.2020.00009.
18. B. Blanchet et al, "*ProVerif: Cryptographic Protocol Verifier in the Formal Model*", https://bblanche.gitlabpages.inria.fr/proverif/
19. *Tamarin Prover.* https://tamarin-prover.github.io
20. Sevdenur Baloglu, "Formal Verification of Verifiability in E-Voting Protocols", Doctoral thesis from Luxembourg University, 2023, https://hdl.handle.net/10993/55700
21. B. Blanchet et al, "*ProVerif with Lemmas, Induction, Fast Subsumption*, *and Much More*", IEEE Symposium on Security & Privacy, Aug 2022, https://youtu.be/AdXiRtEUGZ4
22. Vincent Cheval et al, "Election Verifiability with ProVerif", 5-12-2023, https://inria.hal.science/hal-04177268v2/
23. Copilot Complete Conversation about formal verification tools, https://anonymous.4open.science/r/anonymous_citation-E37B/Copilot_Proverify.pdf
24. Simple court verifiability test of a completely honest process, https://anonymous.4open.science/r/anonymous_citation-E37B/court.pv
25. Running code when exposing receipts to attackers, https://anonymous.4open.science/r/anonymous_citation-E37B/court3.pv
26. The exact text of the run output, https://anonymous.4open.science/r/anonymous_citation-E37B/runresult.txt